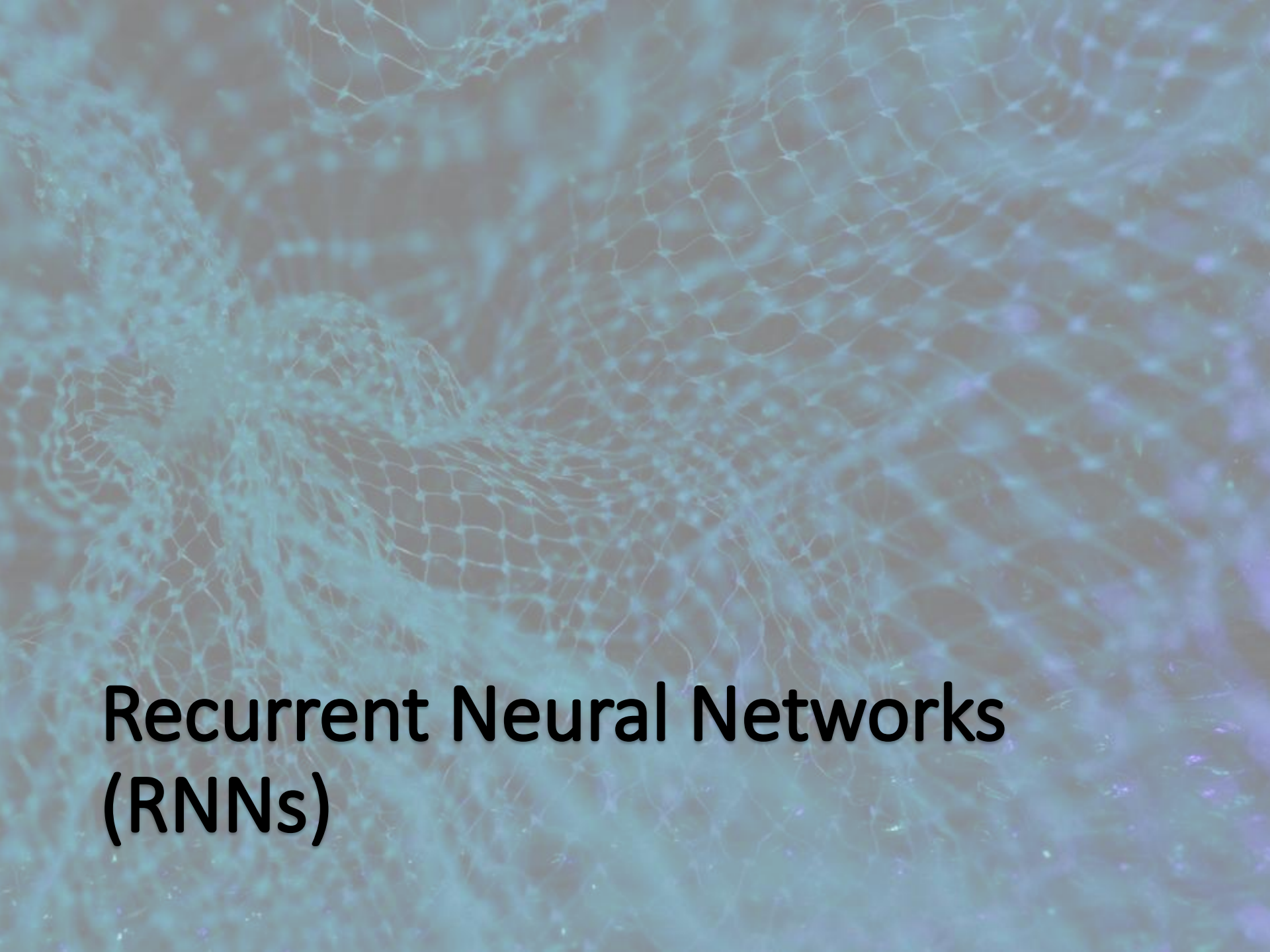




Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs)



RNNs are a type of sequence model.

- A **sequence model** is any model that processes input data as a sequence—i.e., where order matters.
- Examples of sequences: text (a sequence of words), time-series data (a sequence of values over time), speech (a sequence of audio frames), etc.

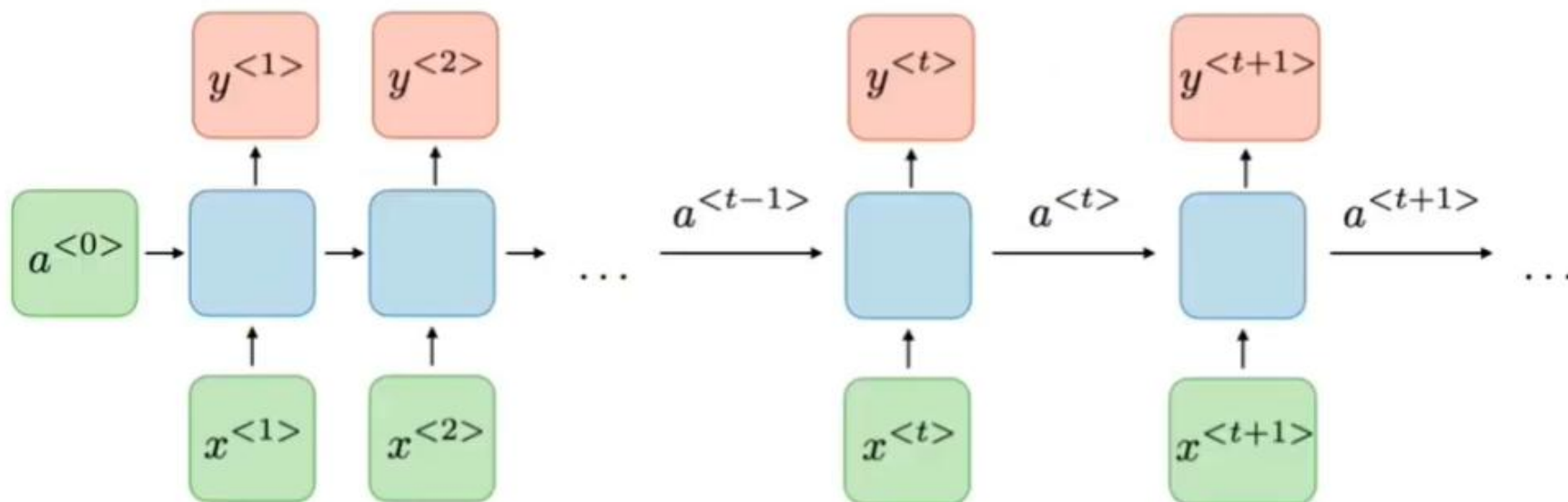
Recurrent Neural Networks (RNNs)

- Recurrent Neural Networks (RNNs) work a bit different from regular neural networks. In neural network the information flows in one direction from input to output.
- However, in RNN information is fed back into the system after each step.
- Think of it like reading a sentence, when you're trying to predict the next word, you don't just look at the current word but also need to remember the words that came before to make accurate guess.

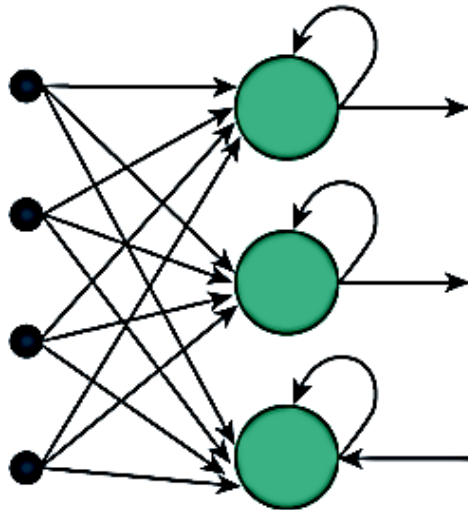
Recurrent Neural Networks (RNNs)

- RNNs allow the network to “remember” past information by feeding the output from one step into next step.
- This helps the network understand the context of what has already happened and make better predictions based on that. For example, when predicting the next word in a sentence the RNN uses the previous words to help decide what word is most likely to come next.

Recurrent Neural Networks (RNNs)



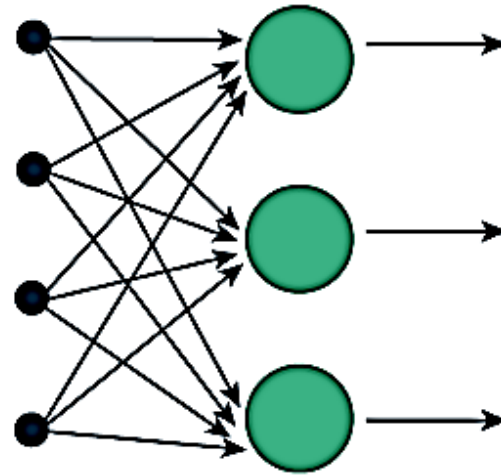
RNN



(a) Recurrent Neural Network

RNNs incorporate loops that allow information **from previous steps to be fed back into the network**. This feedback enables RNNs to remember prior inputs making them ideal for tasks where context is important.

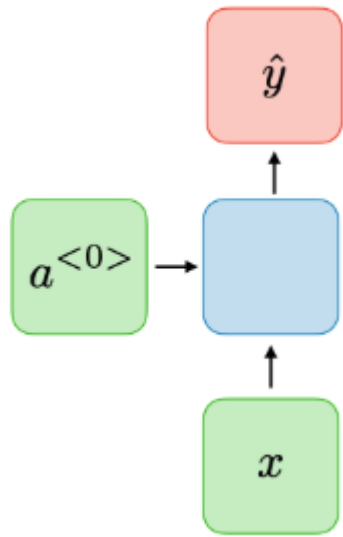
Feedforward NN



(b) Feed-Forward Neural Network

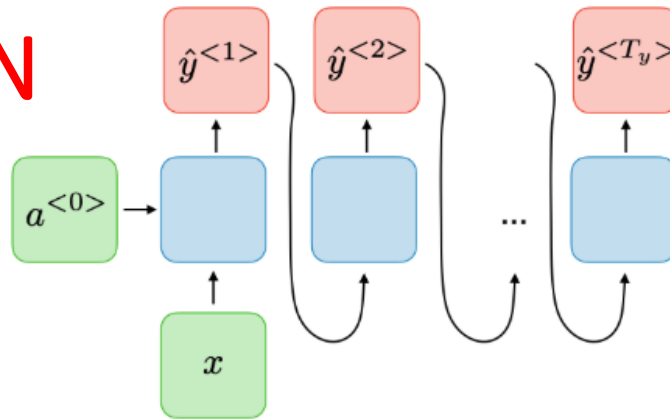
Feedforward Neural Networks (FNNs) process data **in one direction from input to output** without retaining information from previous inputs. This makes them suitable for tasks with **independent inputs** like image classification. However, FNNs struggle with sequential data since they lack memory.

Types of RNN



1. One-to-One RNN

It is used for straightforward classification tasks such as binary classification where no sequential data is involved.

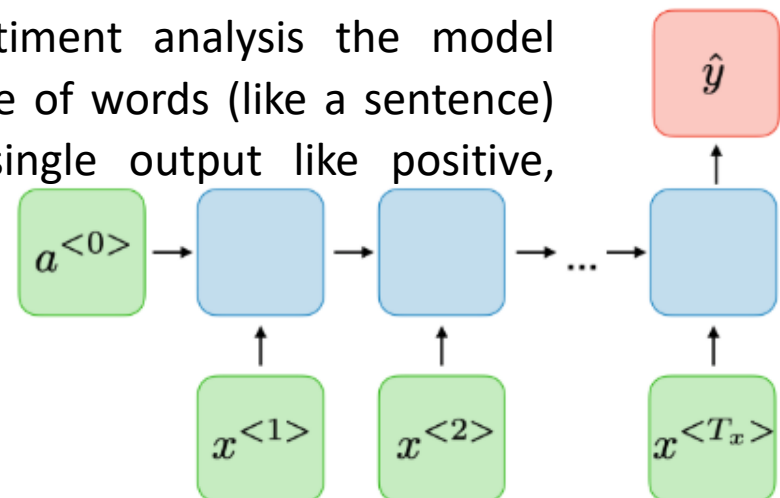


2. One-to-Many RNN

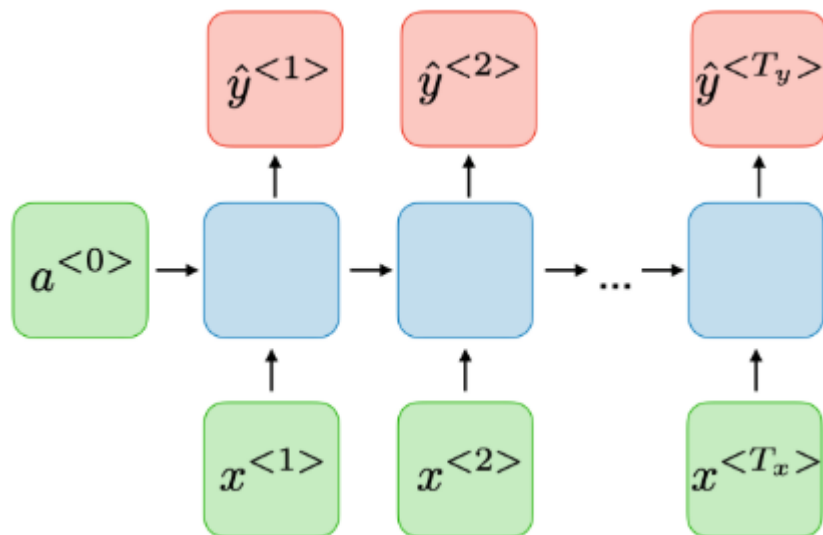
This is useful in tasks where one input triggers a sequence of predictions (outputs). For example, in image captioning a single image can be used as input to generate a sequence of words as a caption.

3. Many-to-One RNN

This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



Types of RNN



Ex: Named Entity Recognition is a subtask of information extraction that seeks to locate and classify entities in text into predefined categories .

Given a sentence:

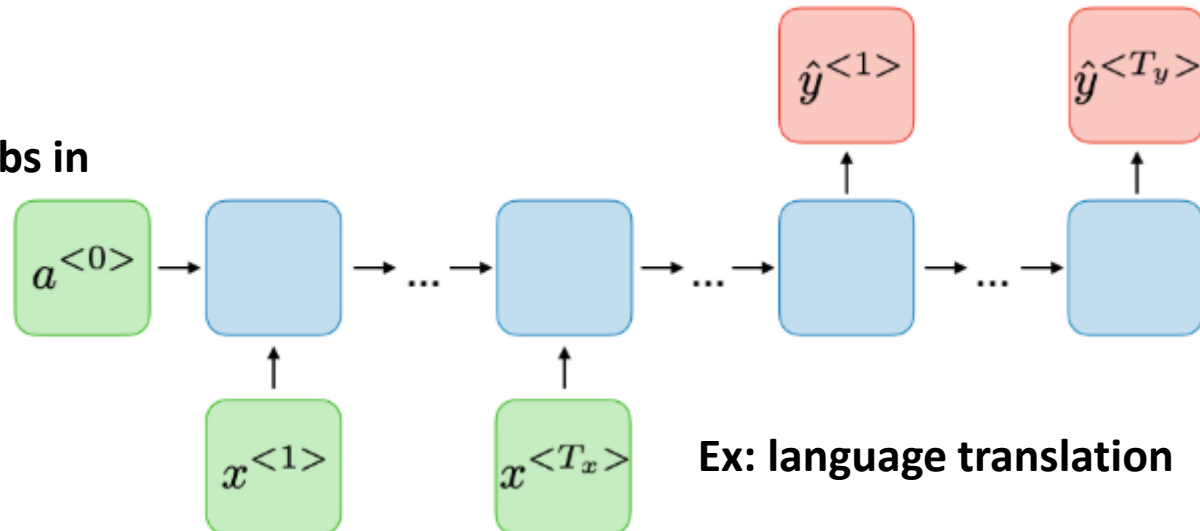
"Apple was founded by Steve Jobs in California."

NER aims to label it like:

"Apple" → Organization

"Steve Jobs" → Person

"California" → Location



Ex: language translation

4. Many-to-Many RNN

The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input, and a corresponding sequence in another language is generated as output.

RNN Applications

RNNs have been successfully applied in several domains, including:

- **Natural Language Processing (NLP):** Language modeling, machine translation, sentiment analysis.
- **Time-Series Forecasting:** Stock price prediction, weather forecasting, economic modeling.
- **Speech and Audio Processing:** Speech recognition, music generation.
- **Video Analysis:** Action recognition, video captioning.